

I2C Inter-Integrated Circuit

- ▷ Synchronous, serial communication protocol
- ▷ Requires only two bidirectional signal lines:
 - SDA: Serial Data Line # 2 wire serial bus protocol
 - SCL: Serial Clock Line # Both are pulled HIGH when bus is idle.
- ▷ Built in device addressing, bidirectional data transfer, and hardware acknowledgement
- ▷ Allows multiple devices to exist on the same exact bus.
 - Each target has an address - to select a device
- ▷ Invented by Philips Semiconductors (NXP Semiconductors)
 - Replaces parallel data buses with a simple two-wire serial interface
- ▷ Short distance communication between integrated circuits on the same PCB or within the same system

Why I2C?

- ▷ A controller wants to communicate with multiple devices but pins are limited
- ▷ It can connect to many I2C device using the same SDA and SCL.

I2C Advantages

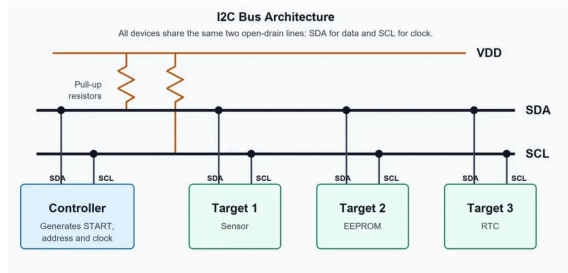
- ▷ Requires only 2 Lines
- ▷ Supports multiple devices
- ▷ ACK/NACK - basic transfer level handshaking
- ▷ Supports multi-controller systems through arbitration
- ▷ Simple and supported by many ICs and microcontrollers.

I2C Limitations

- ▷ Slower than SPI
- ▷ Bus speed and reliability - pull up resistance and bus capacitance
- ▷ Short-distance board level communication

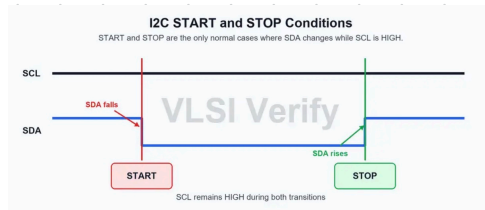
I2C Applications

- ▷ EEPROMs
- ▷ Real-time clocks
- ▷ ADCs and DACs
- ▷ Temp, pressure, humidity and light sensors
- ▷ Accelerometers, gyroscopes and magnetometers

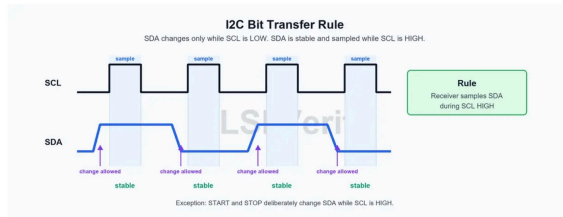


I2C systems can have
 → one controller and multiple devices
 → Multiple controllers - use arbitration to decide which controller owns the bus if two are transmitting at the same time.

SCL and SDA :



- ▷ SDA carries address, data and acknowledgement bits
- ▷ SCL carries clock - generated by the controller
- ▷ One bit is transferred at each clock
- ▷ Data on SDA must be stable while SCL is HIGH. SDA can change only while SCL is LOW.

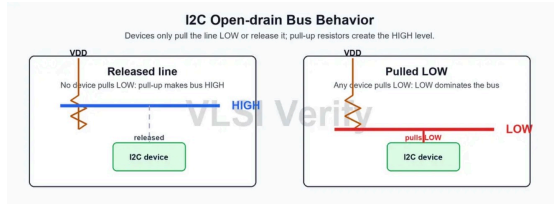


▷ **START**: SDA $\downarrow$ while SCL is HIGH.

▷ **STOP**: SDA $\uparrow$ while SCL is HIGH

Open-drain bus and pull up resistors

- ▷ I2C devices do not drive the bus HIGH, instead they pull SDA or SCL LOW.
- ▷ Open drain or open collector signaling
- ▷ Pull up resistors pull the lines back up to HIGH when no device is active



- Pull up resistor affects rise time, power consumption and max speed bus
- A large resistor reduces current but makes the rising edge slower.
- A small resistor improves rise time but increases current when the line is LOW.
- Commonly used are: - 22k Ω , 47k Ω or 10k Ω

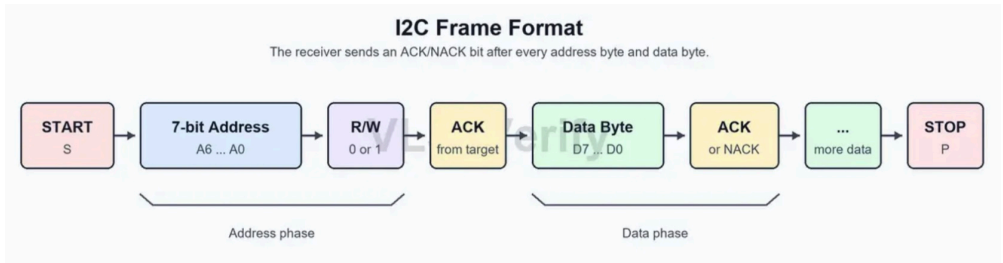
Controller :-

- ▷ Generates START
- ▷ Sends target address
- ▷ Controls clock
- ▷ Ends the transfer using a STOP

Target :-

- ▷ Responds when address is matched.
- ▷ Receives/Transmits data depending on the read/write bit

I2C Frame Format

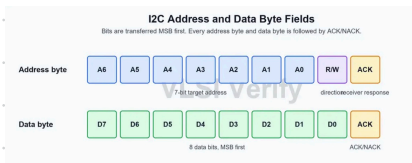


START :

- ▷ Transaction beginning
- ▷ When target device detects **START**, starts listening for an address.

Target Address :-

- ▷ **7 bit address** – more common in embedded system.
- ▷ Also supports 10-bit addressing



Read/Write bit :-

The read/write bit tells the target the direction of transfer.

R/W bit	Operation	Direction
0	Write	Controller sends data to target
1	Read	Target sends data to controller

ACK/NACK bit :-

- ▷ After every byte -

- **ACK** : Receiver pulls **SDA LOW** during the acknowledge clock pulse.
- **NACK** : Receiver leaves **SDA HIGH** during the acknowledge clock pulse

- ▷ **ACK**: Receiver accepted the byte

NACK :

- Byte was not accepted

- Target is unavailable
- Receiver wants to end the transfer

- ▷ For **Write** : Target Ack's each received byte.

For **Read** : Controller Ack's bytes if it wants to continue, NACK the last byte to indicate read is complete.

Data byte :

- ▷ 8 bits at a time, MSB first.
- ▷ ACK/NACK after each byte.
- ▷ No fixed limit on the number of bytes

STOP :

- ▷ Marks the end of a transaction
- ▷ After STOP, bus becomes idle if both SDA and SCL are HIGH.

Repeated START condition :-

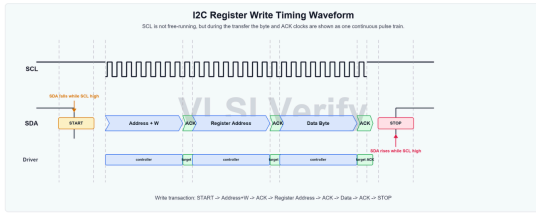
- ▷ A repeated START is a START condition without first sending a STOP. It allows the controller to change direction or address another target while keeping control of the bus.

Read and Write Transactions :-

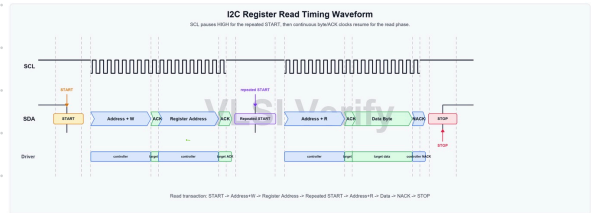
- ▷ Many devices also use internal registers, so a complete communication often includes both a device address and a register address.
- ▷ Device address - select a target address
- ▷ Register address - points to an internal register address inside that device.



Write Operation :-



Read Operation :-



Controller still generates the SCL, the target drives SDA during the data byte.

I2C Modes

Mode	Maximum bit rate	Notes
Standard mode	100 kbit/s	Common for simple low-speed devices
Fast mode	400 kbit/s	Common in many embedded systems
Fast mode plus	1 Mbit/s	Higher speed, stronger bus drive, and tighter timing
High-speed mode	3.4 Mbit/s	Uses a special high-speed transfer sequence
Ultra Fast-mode	5 Mbit/s	Unidirectional mode, not used like normal bidirectional I2C

Suitable when bus has several devices

Firmware needs to read sensor data periodically or update registers more often

Transfers large blocks of data, updates many registers or communicates with devices that need faster service

I2C is required but data rate of fast mode plus is not enough.

Write only transfers

References :

➤ VLSI Verify